

Contents

TktSync API & Partner Integration Contract	1
PART I - SHARED REPRESENTATIONS	8
PART II - PARTNER API	12
PART III - WHITE-LABEL SELECTION API	21
PART IV - ADMISSION API	22
PART V - ADMIN API	23
PART VI - PARTNER INTEGRATION OPERATING RULES	27
PART VII - ERROR MODEL	29
PART VIII - RETRIES, RATE LIMITING, AND FAILURE BEHAVIOR	31
PART IX - SECURITY BOUNDARIES AT THE API CONTRACT	31
PART X - CONTRACT GENERATION AND CLIENTS	32
PART XI - ASYNCHRONOUS NOTIFICATION BOUNDARY	33
PART XII - CROSS-DOCUMENT TRACEABILITY	33
PART XIII - REVIEW FINDINGS AND DRIFT RESOLUTIONS	35
PART XIV - NON-NEGOTIABLE API INVARIANTS	37

TktSync API & Partner Integration Contract

Document status: Governing External API & Partner Integration Contract

Version: 1.0

Date: 20 August 2026

Classification: Confidential

Applies to: TktSync MVP API surfaces and compatible partner integrations

Normative parents, in precedence order:

1. TktSync Platform Process & Policy Standard v1.0
2. TktSync Logical Domain Specification v1.0
3. TktSync System Architecture & Transactional Design Specification v1.0
4. TktSync Relational Data Model & Schema Specification v1.0
5. TktSync Technology Stack Decision Record v1.0

Product basis: TktSync Technical Brief (2026)

1. Purpose

This document defines the stable external behavior of TktSync's HTTP APIs and the integration obligations of ticketing partners.

It specifies:

- API surfaces and versioning;
- authentication and authorization boundaries;
- public identifiers;
- idempotency and retry semantics;
- request and response conventions;
- time and money representation;
- event, layout, and availability reads;

- Reservation creation, modification, checkout protection, payment-failure reporting, confirmation, release, and recovery;
- white-label selection-session behavior;
- ticket and QR credential retrieval;
- admission/scanning behavior;
- administrative command surfaces;
- partner reporting and audit access;
- business error semantics;
- compatibility expectations;
- security constraints relevant to the API boundary;
- traceability to all governing TktSync specifications.

This contract does **not** redefine database transactions, state-machine semantics, inventory ownership, or payment responsibility. It exposes the already-approved TktSync behavior through a stable protocol.

2. Governing Principle

The API is a contract over the domain, not a contract over database tables.

Clients **MUST** reason in terms of:

- Event;
- availability;
- Reservation;
- CheckoutAttempt;
- Sale;
- TicketEntitlement;
- credential;
- Admission;
- block/allocation;
- partner authorization.

Database table names, internal claim rows, lock ordering, counter buckets, and outbox rows **MUST NOT** become public integration concepts unless a future version explicitly promotes them to the external domain.

3. API Surfaces

TktSync exposes four logical HTTP surfaces from the same authoritative Go Core API.

- **Partner API** - consumed by ticketing-platform servers; provides availability, Reservations, checkout protection, confirmation, release, ticket retrieval, and Partner reporting.
- **Selection API** - consumed by the TktSync white-label buyer selector; provides contextual availability and the buyer's own pre-payment Reservation operations.
- **Admission API** - consumed by the authorized scanner web application; provides credential validation and authoritative admission.
- **Admin API** - consumed by authorized TktSync/event-owner operators; provides Venue, Event, pricing, inventory-restriction, lifecycle, ticket, and reporting operations.

The surfaces are authorization boundaries, not separate authoritative services.

3.1 API Scope Exclusions

The MVP API does not provide platform authority for:

- payment processing or refunds;
- Partner service-fee calculation;
- CRM/customer messaging;
- dynamic pricing;
- native mobile-only workflows;
- self-service Partner onboarding;

- enterprise analytics;
- multilingual content management;
- secondary-market ticket transfer/resale.

These capabilities MUST NOT be simulated by weakening inventory, confirmation, Ticket, or admission semantics.

4. Base Path and Transport

The first major version SHALL use:

/api/v1

Representative logical prefixes:

/api/v1/partner

/api/v1/selection

/api/v1/admission

/api/v1/admin

All production requests MUST use HTTPS.

Plain HTTP MUST NOT be accepted for authenticated production traffic.

JSON APIs SHALL use:

Content-Type: application/json

Accept: application/json

Credential payload/image responses MAY use an appropriate non-JSON media type where explicitly documented.

5. Versioning and Compatibility

5.1 Major Versions

Breaking behavioral or representation changes require a new major API path such as /api/v2.

5.2 Backward-Compatible Changes

Within /api/v1, TktSync MAY:

- add optional response fields;
- add new read-only endpoints;
- add optional request fields whose omission preserves prior behavior;
- add new business error codes for newly introduced optional behavior;
- add new enum values only when existing clients can safely treat unknown values as non-actionable.

5.3 Request Strictness

Unknown request fields SHOULD be rejected with VALIDATION_ERROR rather than silently ignored on authoritative mutation endpoints. This prevents misspelled or unsupported intent from being silently dropped.

5.4 Response Extensibility

Clients MUST ignore unknown response fields.

Clients MUST NOT fail solely because a response contains an additional field.

5.5 Unknown States

A client receiving an unknown canonical state MUST NOT infer that the state is equivalent to AVAIL - ABLE, HELD, CONFIRMED, or any other known state. The safe behavior is to stop the irreversible action, resynchronize, and upgrade integration logic if required.

6. Public Identifiers

All resource identifiers are opaque strings.

Examples in this document may use illustrative prefixes such as:

```
evt_...  
res_...  
sale_...  
tkr_...  
inv_...  
ga_...  
chk_...
```

These prefixes are illustrative and MUST NOT be parsed for business meaning unless the production OpenAPI contract explicitly fixes them.

Clients MUST NOT:

- infer database type from an identifier;
 - derive one resource identifier from another;
 - assume lexical ordering;
 - assume creation time from an identifier;
 - expose internal UUID semantics as business behavior.
-

7. Authentication Classes

7.1 Partner API

Partner server-to-server requests SHALL authenticate using:

Authorization: Bearer <partner-credential>

The exact credential cryptographic format is defined by the Security/Auth specification.

Authentication resolves a PartnerCredential and Partner identity. Authorization separately evaluates Partner state and PartnerEventAccess.

A valid credential does not imply access to every Event.

7.2 Selection API

Selection API requests SHALL use an opaque, time-bounded BuyerSelectionSession capability.

Authorization: Bearer <selection-capability>

The capability is scoped to:

- one Partner;
- one Event;
- one buyer/session context;
- the limited selection operations defined in this contract.

7.3 Admin and Admission APIs

Human admin/scanner requests SHALL use an authenticated human bearer session whose role and Event scope are evaluated on every command.

The exact human token format is deferred to the Security/Auth specification.

8. Authorization Is Separate From Resource Knowledge

Knowing any of the following does not confer authority:

- Event ID;
- Reservation ID;
- Sale ID;
- Ticket ID;
- Reservation continuation token;
- QR credential ID.

Every protected command **MUST** independently verify the authenticated actor and applicable Event/Partner scope.

9. Reservation Continuation Token

A successful Reservation creation returns an opaque continuation token.

Normal owner-side Reservation mutations **SHOULD** require both:

1. the appropriate authenticated Partner or BuyerSelectionSession authority; and
2. the Reservation continuation token.

The token **SHALL** be sent in a header rather than in a URL:

X-TktSync-Reservation-Token: <opaque-token>

The token **MUST NOT** appear in query strings or routine logs.

Responses containing Reservation continuation material **SHOULD** include Cache-Control: no-store.

Possession of the token alone is not authorization.

9.1 Idempotent Replay Requirement

If a Reservation-creation response is lost, retrying the same request with the same idempotency identity **MUST** allow the caller to recover the same logical Reservation and a usable continuation token.

The Security/Auth implementation therefore **MUST** provide replay-safe token recovery without requiring ordinary plaintext-token storage. Acceptable implementation strategies include deterministic cryptographic derivation or protected encrypted replay material. The exact mechanism is defined later; the API behavior is fixed here.

10. Idempotency

10.1 Required Header

Every externally retrievable state-changing request **MUST** include:

Idempotency-Key: <caller-generated-key>

This applies at minimum to:

- selection-session creation;
- Reservation creation;
- Reservation modification;
- begin checkout;
- definitive payment-failure reporting;
- release;
- confirmation;
- administrative lifecycle commands;

- block/allocation mutations;
- ticket void;
- credential reissue;
- inventory re-release;
- admission scan;
- admission correction/override.

10.2 Key Semantics

The key is scoped by:

- authenticated caller security scope;
- operation type;
- exact idempotency-key value.

10.3 Same Key, Same Intent

A retry with the same normalized request **MUST** return the same logical result.

For a previously successful operation, retry **MUST NOT** create a second business effect.

10.4 Same Key, Different Intent

Reusing the same idempotency identity with materially different request content **MUST** return:

IDEMPOTENCY_CONFLICT

10.5 Lost Response

A client timeout is an unknown outcome, not proof of failure.

The client **MUST** retry the same logical command with the same idempotency key or read authoritative status.

10.6 Token/Credential-Producing Operations

Any operation whose first response includes opaque continuation or credential material **MUST** support recovery of a usable equivalent for the same successful logical operation after response loss.

This requirement applies without changing the one-logical-result rule.

11. Request Correlation

Clients **MAY** supply:

X-Request-Id: <opaque-correlation-id>

TktSync **SHALL** return a request/correlation identifier on every response.

If the supplied identifier is invalid or absent, TktSync generates one.

Request correlation is not idempotency.

12. Time Representation

All API timestamps **SHALL** use RFC 3339 / ISO 8601 timestamps with an explicit offset, normally UTC Z.

Example:

2026-08-20T20:45:31.428Z

Client clocks are advisory.

Server-authoritative deadline fields include:

- `hold_expires_at`;
- `checkout_expires_at`;
- `payment_retry_expires_at`;
- `reconciliation_expires_at`;
- `max_lifetime_at`.

Responses containing active deadlines SHOULD also contain `server_time` so a client can render a countdown without treating its clock as authoritative.

13. Money Representation

Money SHALL be represented as:

```
{
  "amount_minor": 5000000,
  "currency": "NGN"
}
```

Rules:

- `amount_minor` is an integer;
- `currency` is an uppercase three-letter currency code;
- floating-point monetary values MUST NOT be used;
- Partner service fees are not part of TktSync inventory pricing;
- one Reservation carries one transaction currency;
- all Reservation items must match that currency.

If selected offers cannot form one coherent Reservation currency, the hold is rejected with `CURRENCY_MISMATCH`.

14. Pagination

List endpoints SHOULD use cursor pagination.

Representative response:

```
{
  "items": [],
  "next_cursor": "opaque-or-null"
}
```

Clients MUST treat cursors as opaque.

Cursor values MUST NOT be interpreted as database offsets or IDs.

Availability and floor-plan endpoints MAY use section/filter partitioning instead of generic pagination where a complete visual section is the required user experience.

15. Success and Error Response Conventions

Successful resource endpoints return the resource or command result directly.

Errors use one envelope:

```
{
  "error": {
    "code": "INVENTORY_UNAVAILABLE",
    "message": "One or more requested inventory items are no longer available.",
    "request_id": "req_...",
  }
}
```

```

    "details": {
      "inventory_ids": ["inv_..."]
    }
  }
}

```

message is human-readable but not stable integration logic.

error.code is the machine-readable recovery signal.

16. HTTP Status Mapping

HTTP status describes transport-level class; business error code describes recovery behavior.

Condition	Typical HTTP status
Invalid JSON / validation failure	400
Missing/invalid authentication	401
Authenticated but not authorized	403
Resource not visible to caller	404
State/concurrency/business conflict	409
Rate limited	429
Authority temporarily unavailable	503
Unexpected server failure	500

Clients MUST NOT parse message text to determine recovery behavior.

PART I - SHARED REPRESENTATIONS

17. Event Summary

Representative Event response:

```

{
  "id": "evt_...",
  "name": "Championship Night",
  "state": "ON_SALE",
  "starts_at": "2026-09-12T18:00:00Z",
  "sales_open_at": "2026-08-20T09:00:00Z",
  "sales_close_at": "2026-09-12T17:30:00Z",
  "admission_open_at": "2026-09-12T16:00:00Z",
  "admission_close_at": "2026-09-12T22:00:00Z"
}

```

Canonical states:

- DRAFT;
- ON_SALE;
- PAUSED;
- SALES_CLOSED;
- COMPLETED;
- CANCELLED.

Partner-visible event details MUST NOT expose another Partner's configuration or private transaction data.

18. Event Layout Representation

The Event layout endpoint returns the event-specific frozen buyer-facing floor plan, not the mutable reusable Venue template.

It MAY include:

- sections;
- rows;
- tables;
- reserved inventory IDs and labels;
- GA pool IDs and labels;
- buyer-facing geometry;
- orientation/stage/ring/field metadata;
- pricing display references where appropriate.

The exact third-party floor-plan engine's internal model MUST NOT become the Partner API's stable identity model.

Clients SHOULD cache layout structure separately from rapidly changing availability.

19. Availability Offer Model

Availability is a caller-contextual read model.

A returned offer means:

The caller may currently attempt to acquire this inventory under these displayed TktSync-controlled commercial terms.

It does **not** mean the inventory has been reserved.

Each acquirable commercial selection is represented by an opaque offer_id.

The offer_id encapsulates the caller-contextual acquisition source and pricing context without exposing internal Allocation IDs.

An offer MAY become stale before hold creation.

20. Reserved-Seat Availability

Representative entry:

```
{
  "inventory_id": "inv_...",
  "section_id": "sec_...",
  "row": "A",
  "seat": "12",
  "sellability": "AVAILABLE",
  "offer": {
    "offer_id": "off_...",
    "price": {
      "amount_minor": 5000000,
      "currency": "NGN"
    }
  }
}
```

Unavailable seats MAY be returned for visual seat-map rendering:

```
{
  "inventory_id": "inv_...",
  "sellability": "UNAVAILABLE"
}
```

The API MUST NOT disclose whether an unavailable seat is held by another Partner, in checkout, allocated to another channel, blocked for production, sold, or otherwise unavailable unless the caller has administrative authority.

21. GA Availability

A GA pool may expose more than one eligible acquisition offer to the same Partner when TktSync needs to preserve an explicit source-specific acquisition choice between shared availability and the Partner's eligible channel Allocation.

An Allocation changes acquisition eligibility/source; under the governing policy it does not introduce a different price tier from the underlying Event GA pool.

Representative response:

```
{
  "inventory_id": "ga_...",
  "name": "Floor",
  "offers": [
    {
      "offer_id": "off_shared_...",
      "available_quantity": 80,
      "price": {
        "amount_minor": 2000000,
        "currency": "NGN"
      }
    },
    {
      "offer_id": "off_channel_...",
      "available_quantity": 20,
      "price": {
        "amount_minor": 2000000,
        "currency": "NGN"
      }
    }
  ]
}
```

Internal source names such as Allocation IDs do not need to be exposed.

A hold request selects a specific offer. TktSync MUST NOT silently substitute a different offer/source if the selected one can no longer be acquired.

22. Reservation Representation

Representative Reservation:

```
{
  "id": "res_...",
  "event_id": "evt_...",
  "status": "HELD",
  "currency": "NGN",
  "partner_customer_ref": "customer-42",
  "partner_order_ref": "cart-923",
  "hold_expires_at": "2026-08-20T20:50:00Z",
  "payment_retry_expires_at": null,
  "reconciliation_expires_at": null,
  "max_lifetime_at": "2026-08-20T20:56:00Z",
  "server_time": "2026-08-20T20:46:10Z",
  "items": [],
  "total": {
```

```

    "amount_minor": 9000000,
    "currency": "NGN"
  }
}

```

Canonical statuses:

- HELD;
- COMMITTING;
- PAYMENT_RETRY;
- RECONCILING;
- CONFIRMED;
- RELEASED;
- EXPIRED.

The API MUST NOT replace these canonical machine states with vague integration states such as PENDING, PROCESSING, or FAILED.

User interfaces MAY map them to friendlier copy.

23. Reservation Item Representation

Representative reserved item:

```

{
  "id": "ritem_...",
  "inventory_kind": "RESERVED",
  "inventory_id": "inv_...",
  "quantity": 1,
  "display": {
    "section": "VIP",
    "row": "A",
    "seat": "12"
  },
  "price": {
    "amount_minor": 5000000,
    "currency": "NGN"
  }
}

```

Representative GA item:

```

{
  "id": "ritem_...",
  "inventory_kind": "GA",
  "inventory_id": "ga_...",
  "quantity": 2,
  "display": {
    "section": "Floor"
  },
  "price": {
    "amount_minor": 2000000,
    "currency": "NGN"
  }
}

```

The price is the TktSync-controlled snapshot captured when the item was acquired.

Partner fees are not included.

24. Sale Representation

```
{
  "id": "sale_...",
  "reservation_id": "res_...",
  "event_id": "evt_...",
  "partner_order_ref": "ORD-9273",
  "partner_payment_ref": "PAY-91827",
  "currency": "NGN",
  "confirmed_at": "2026-08-20T20:48:31Z",
  "items": []
}
```

A Sale is immutable commercial history.

Ticket voiding or later inventory re-release does not rewrite the Sale.

25. Ticket Representation

```
{
  "id": "tkr_...",
  "event_id": "evt_...",
  "status": "ACTIVE",
  "inventory_kind": "RESERVED",
  "inventory": {
    "inventory_id": "inv_...",
    "section": "VIP",
    "row": "A",
    "seat": "12"
  },
  "credential": {
    "id": "cred_...",
    "status": "ACTIVE"
  }
}
```

Ticket status:

- ACTIVE;
- VOIDED.

QR credential identity remains separate.

PART II - PARTNER API

26. Partner Event Endpoints

```
GET /api/v1/partner/events/{event_id}
GET /api/v1/partner/events/{event_id}/layout
GET /api/v1/partner/events/{event_id}/availability
```

Only Events covered by active PartnerEventAccess are visible.

A disabled Event access grant MUST NOT be bypassed by knowing the Event ID.

27. GET /partner/events/{event_id}/availability

The response SHALL include:

- Event ID;

- as_of timestamp;
- server_time;
- optional opaque/monotonic revision freshness value;
- caller-contextual reserved-seat sellability;
- caller-contextual GA offers;
- TktSync-controlled pricing required to make a hold selection.

Example:

```
{
  "event_id": "evt_...",
  "as_of": "2026-08-20T20:45:00Z",
  "server_time": "2026-08-20T20:45:01Z",
  "revision": "9281",
  "reserved_units": [],
  "ga_pools": []
}
```

27.1 Contractual Warning

Availability is informational.

A displayed AVAILABLE item or quantity is not owned until POST /reservations succeeds.

27.2 Caching

The endpoint MAY be cached for short periods.

The API SHOULD expose freshness metadata sufficient for clients to understand the snapshot age.

Caching MUST NOT change hold semantics.

28. POST /partner/reservations

Creates one atomic Reservation.

Required headers:

Authorization: Bearer <partner-credential>

Idempotency-Key: <key>

Representative request:

```
{
  "event_id": "evt_...",
  "partner_customer_ref": "customer-928",
  "partner_order_ref": "cart-727",
  "buyer_session_ref": "browser-session-11",
  "items": [
    {
      "offer_id": "off_seat_a12",
      "quantity": 1
    },
    {
      "offer_id": "off_ga_floor",
      "quantity": 2
    }
  ]
}
```

Rules:

- at least one item is required;
- reserved-seat offer quantity MUST be 1;
- GA quantity MUST be positive;

- the complete request is all-or-nothing;
- all items must form one coherent transaction currency;
- selected offers are revalidated inside the authoritative hold transaction;
- no inventory is silently substituted;
- the Reservation lifetime is server-controlled;
- Partner references are opaque correlation values and are not substitutes for idempotency.

28.1 Success

```
{
  "reservation": {
    "id": "res_...",
    "event_id": "evt_...",
    "status": "HELD",
    "currency": "NGN",
    "hold_expires_at": "2026-08-20T20:50:00Z",
    "max_lifetime_at": "2026-08-20T20:56:00Z",
    "server_time": "2026-08-20T20:45:00Z",
    "items": [],
    "total": {
      "amount_minor": 9000000,
      "currency": "NGN"
    }
  },
  "reservation_token": "opaque-continuation-token"
}
```

The token is returned only to an authorized caller, must not be placed in URLs, and must be recoverable on idempotent replay.

29. Hold Failure Semantics

If any item cannot be acquired, the request fails atomically.

Representative business errors:

- INVENTORY_UNAVAILABLE;
- INSUFFICIENT_GA_QUANTITY;
- INVENTORY_NOT_ELIGIBLE_FOR_PARTNER;
- EVENT_NOT_ON_SALE;
- PARTNER_DISABLED;
- PARTNER_EVENT_ACCESS_DISABLED;
- CURRENCY_MISMATCH.

A failure **MUST NOT** leave a partial Reservation.

The Partner **MAY** refresh availability and submit a new customer-approved intent using a new idempotency key.

30. GET /partner/reservations/{reservation_id}

Returns the authoritative Reservation visible to the owning Partner.

This endpoint is the primary recovery mechanism after network ambiguity.

It does not require the API to disclose the raw Reservation continuation token.

A Partner **MUST NOT** read another Partner's private Reservation by guessing an ID.

31. PATCH /partner/reservations/{reservation_id}

Atomically modifies a Reservation while its effective state is HELD.

Required headers:

Authorization: Bearer <partner-credential>

X-TktSync-Reservation-Token: <token>

Idempotency-Key: <key>

Representative request:

```
{
  "remove_item_ids": ["ritem_old"],
  "adjust_quantities": [
    {
      "reservation_item_id": "ritem_ga",
      "new_quantity": 3
    }
  ],
  "add_items": [
    {
      "offer_id": "off_new_seat",
      "quantity": 1
    }
  ]
}
```

Rules:

- only HELD Reservations are modifiable;
 - additions/increases must be authoritatively reacquired;
 - all new acquisitions are validated before removals are committed;
 - if any addition/increase fails, the original valid composition remains unchanged;
 - retained items keep their original price snapshot;
 - newly added/increased inventory snapshots current applicable terms;
 - modification MUST NOT reset max_lifetime_at;
 - Event PAUSED or SALES_CLOSED permits only non-expanding changes that release inventory.
-

32. POST /partner/reservations/{reservation_id}/checkout

Transitions an eligible Reservation into protected checkout.

This endpoint MUST succeed **before** the Partner begins an irreversible or potentially chargeable payment attempt.

Required headers:

Authorization: Bearer <partner-credential>

X-TktSync-Reservation-Token: <token>

Idempotency-Key: <key>

Request body MAY be empty.

Eligible Reservation states are HELD and PAYMENT_RETRY. A PAYMENT_RETRY transition creates a new protected CheckoutAttempt within the remaining bounded Reservation lifetime.

If another CheckoutAttempt is already active, the command returns CHECKOUT_ALREADY_ACTIVE.

Success:

```
{
  "reservation_id": "res_...",
  "status": "COMMITTING",
  "checkout_attempt": {
    "id": "chk_...",
    "status": "ACTIVE",
```

```

    "checkout_expires_at": "2026-08-20T20:52:00Z"
  },
  "server_time": "2026-08-20T20:50:31Z"
}

```

32.1 Event State

An already valid Reservation MAY begin checkout while Event state is:

- ON_SALE;
- PAUSED;
- SALES_CLOSED;

provided its own authorization window remains valid.

CANCELLED and COMPLETED reject new checkout protection.

32.2 Partner Obligation

If this command does not succeed, the Partner MUST NOT intentionally charge the customer for that attempt.

33. Payment Responsibility Boundary

TktSync does not:

- authorize payment;
- capture payment;
- settle payment;
- calculate Partner service fees;
- perform refunds;
- infer success from elapsed time.

The Partner owns all payment-provider interaction.

TktSync only protects inventory and accepts authoritative confirmation/release signals under the approved state rules.

34. POST /partner/reservations/{reservation_id}/payment-failure

Reports that the Partner has established a **definitive** payment failure or definite non-charge outcome for a specific CheckoutAttempt.

Required headers:

Authorization: Bearer <partner-credential>

X-TktSync-Reservation-Token: <token>

Idempotency-Key: <key>

Representative request:

```

{
  "checkout_attempt_id": "chk_...",
  "partner_payment_ref": "PAY-91827",
  "failure_code": "CARD_DECLINED",
  "requested_disposition": "RETRY"
}

```

requested_disposition values:

- RETRY;
- RELEASE.

TktSync may reject RETRY if the configured retry budget/lifetime has been exhausted.

Possible successful results:

```
{
  "reservation_id": "res_...",
  "status": "PAYMENT_RETRY",
  "payment_retry_expires_at": "2026-08-20T20:53:30Z"
}
```

or:

```
{
  "reservation_id": "res_...",
  "status": "RELEASED"
}
```

34.1 Unknown Payment Outcome

This endpoint MUST NOT be called merely because the payment provider timed out or the Partner has not yet determined the outcome.

Unknown outcomes are handled by RECONCILING semantics.

A false definitive-failure signal can cause customer harm and is a Partner contract violation.

35. Checkout Timeout and Reconciliation

The Partner does not call an endpoint to force RECONCILING merely to extend inventory ownership.

When protected checkout becomes uncertain at the authoritative deadline, TktSync moves the transaction into reconciliation according to governing policy.

The Partner observes this through GET /reservations/{id}.

During RECONCILING:

- inventory remains unavailable to others;
 - a valid delayed confirmation MAY still succeed before reconciliation expiry;
 - a definitive payment failure MAY be reported;
 - ordinary voluntary release without establishing payment safety is rejected.
-

36. POST /partner/reservations/{reservation_id}/confirm

Confirms one commercial Sale.

Required headers:

Authorization: Bearer <partner-credential>

X-TktSync-Reservation-Token: <token>

Idempotency-Key: <key>

Representative request:

```
{
  "checkout_attempt_id": "chk_...",
  "partner_order_ref": "ORD-9273",
  "partner_payment_ref": "PAY-91827"
}
```

Rules:

- Reservation must be validly COMMITTING or RECONCILING;
- referenced CheckoutAttempt must correspond to the accepted transaction attempt;
- confirmation is idempotent;
- confirmation creates exactly one Sale;

- confirmation creates all required TicketEntitlements;
- reserved item quantity 1 creates one root TicketEntitlement;
- GA quantity N creates N independently admissible root TicketEntitlements;
- confirmation preserves the Reservation's commercial snapshots;
- no new Sale may be created after Event cancellation has already committed.

36.1 Success

```
{
  "reservation_id": "res_...",
  "status": "CONFIRMED",
  "sale": {
    "id": "sale_...",
    "confirmed_at": "2026-08-20T20:51:40Z",
    "partner_order_ref": "ORD-9273",
    "partner_payment_ref": "PAY-91827"
  },
  "tickets": [
    {
      "id": "tk_...",
      "status": "ACTIVE",
      "credential_id": "cred_..."
    }
  ]
}
```

The confirmation response need not expose raw QR payload material inline. Authorized credential retrieval is separately available.

36.2 Retry With Same Idempotency Identity

Returns the same logical Sale and Ticket identities.

36.3 Different Confirmation Identity After Sale Exists

If a different logical confirmation request is submitted after the Reservation is already confirmed, TktSync returns ALREADY_CONFIRMED and SHOULD include the existing Sale reference where authorization permits.

37. POST /partner/reservations/{reservation_id}/release

Voluntarily releases an eligible unconfirmed Reservation.

Required headers:

Authorization: Bearer <partner-credential>

X-TktSync-Reservation-Token: <token>

Idempotency-Key: <key>

Normal eligible states include:

- HELD;
- PAYMENT_RETRY where payment safety is already established.

For COMMITTING or RECONCILING, the Partner SHOULD use the definitive payment-failure path. If payment safety has not been established, ordinary release returns PAYMENT_STATUS_UNCERTAIN.

The Partner MUST NOT supply an inventory return destination.

TktSync performs source-aware restoration internally.

A confirmed Sale cannot be released through this endpoint.

38. Sale Retrieval

GET /api/v1/partner/sales/{sale_id}
GET /api/v1/partner/events/{event_id}/sales

Only the owning Partner's commercial records are visible through Partner scope.

List endpoints are cursor-paginated.

39. Ticket Retrieval

GET /api/v1/partner/tickets/{ticket_id}
GET /api/v1/partner/sales/{sale_id}/tickets

Partner access is limited to Tickets originating from that Partner's commercial Sale.

Non-public issuance Tickets are not exposed as Partner commercial Tickets unless a future explicit policy grants access.

40. QR Credential Retrieval

Because Partners own customer communication, an authorized Partner must be able to obtain the active QR representation for its own active Ticket.

GET /api/v1/partner/tickets/{ticket_id}/credential

Response headers SHOULD include:

Cache-Control: no-store

Representative response:

```
{
  "ticket_id": "tkt_...",
  "credential_id": "cred_...",
  "status": "ACTIVE",
  "qr_payload": "opaque-renderable-value"
}
```

The exact QR cryptographic representation is deferred to the Security/Auth specification.

40.1 Recovery Requirement

Credential retrieval **MUST** be reproducible for the currently active credential without creating a new Ticket identity.

The implementation **MAY** derive the opaque QR payload deterministically from protected server secrets and credential identity, or use another secure recoverable design.

This preserves Partner recovery after a lost confirmation response while remaining compatible with database storage that does not require ordinary plaintext credential persistence.

If the Ticket is VOIDED or no active credential is valid, the endpoint **MUST NOT** manufacture a new credential as a side effect of a read. It returns the corresponding Ticket/credential business state.

40.2 Credential Reissue

Credential rotation is a state-changing operation and is **not** implied by merely retrieving the current credential.

41. Partner Ticket Operations

For Tickets originating from the authenticated Partner's commercial Sales, the Partner MAY use the ticket operations authorized by the Logical Domain Specification.

Representative endpoints:

```
POST /api/v1/partner/tickets/{ticket_id}/void
POST /api/v1/partner/tickets/{ticket_id}/credentials/reissue
POST /api/v1/partner/tickets/{ticket_id}/inventory/re-release
```

All are state-changing operations and require Idempotency-Key.

41.1 Void

voids the TicketEntitlement and revokes the active QR credential.

A Partner MUST NOT use Ticket void to imply that payment has been refunded; refund processing remains Partner-owned.

Ticket void does not automatically make inventory available.

41.2 Credential Reissue

Rotates the active QR credential while preserving Ticket identity.

The previous credential becomes invalid and the Partner can retrieve the replacement credential through the credential endpoint.

41.3 Inventory Re-release

Partner-requested re-release is permitted only when the Event policy explicitly allows the owning Partner to request it.

The command remains subject to TktSync inventory guards and source/destination rules.

If Event policy does not permit Partner-directed re-release, the request returns NOT_AUTHORIZED or a more specific policy error.

42. Partner Reporting

The MVP SHALL provide Partner-scoped reporting sufficient to reconcile the Partner's own activity without exposing another Partner's private transaction details.

Representative endpoints:

```
GET /api/v1/partner/events/{event_id}/reports/inventory
GET /api/v1/partner/events/{event_id}/reports/sales
GET /api/v1/partner/events/{event_id}/activity
```

Partner reporting MAY include:

- current caller-contextual availability summary;
- Partner active Reservations;
- confirmed Sale counts/quantities;
- Ticket status summary;
- Partner-scoped operational activity.

It MUST distinguish SOLD from merely held/committing/reconciling inventory.

PART III - WHITE-LABEL SELECTION API

43. Partner Creates a Selection Session

A Partner without its own seat-selection UI creates a BuyerSelectionSession server-to-server.

POST /api/v1/partner/selection-sessions

Required:

- Partner authentication;
- active PartnerEventAccess;
- idempotency key.

Representative request:

```
{
  "event_id": "evt_...",
  "buyer_session_ref": "buyer-browser-881",
  "return_url": "https://partner.example/checkout/return"
}
```

return_url MUST be HTTPS and MUST match a Partner pre-registered allowed return destination.

Representative response:

```
{
  "selection_session_id": "sels_...",
  "selection_url": "https://select.tktsync.example/s/opaque",
  "expires_at": "2026-08-20T21:10:00Z"
}
```

selection_url is opaque. Partners MUST NOT parse or construct it.

The exact transport of the selection capability inside the opaque URL/bootstrap flow is a Security/Auth implementation detail. Responses containing selection bootstrap/capability material SHOULD use Cache-Control: no-store.

44. Selection API Allowed Operations

The white-label selector MAY:

```
GET    /api/v1/selection/session
GET    /api/v1/selection/event
GET    /api/v1/selection/layout
GET    /api/v1/selection/availability
POST   /api/v1/selection/reservations
GET    /api/v1/selection/reservations/{reservation_id}
PATCH /api/v1/selection/reservations/{reservation_id}
POST   /api/v1/selection/reservations/{reservation_id}/release
```

The Selection API MUST NOT permit:

- begin checkout;
- payment reporting;
- Sale confirmation;
- ticket void;
- Event configuration;
- inventory restriction management;
- unrestricted Partner API access.

45. Selection Reservation Semantics

Selection Reservation creation/modification uses the same authoritative hold semantics as the Partner API.

It is not a weaker inventory path.

The Reservation remains owned by the Partner associated with the BuyerSelectionSession.

The buyer capability grants only the narrowly scoped actions permitted in Section 44. Selection-side modification and voluntary release are permitted only while the Reservation remains effectively HELD; the buyer-facing selector cannot operate payment/protected-checkout states.

46. White-Label Handoff to Partner Checkout

After a successful hold, the selector returns the buyer to the Partner's registered checkout surface with the Reservation identity and continuation token required for server-side continuation.

The default handoff SHOULD use an HTTPS browser form POST rather than placing the continuation token in a query string.

Representative handoff fields:

```
reservation_id
reservation_token
```

The Partner server then authenticates with its Partner credential and calls the Partner API to begin checkout.

The selector itself never performs payment.

PART IV - ADMISSION API

47. Admission Authentication

Admission API access requires an authenticated event-scoped human principal with scanner authority.

A scanner credential for Event A MUST NOT authorize admission operations for Event B.

48. POST /admission/scans

Creates one authoritative ScanAttempt and, if valid, one Admission.

POST /api/v1/admission/scans

Required:

Authorization: Bearer <scanner-session>

Idempotency-Key: <scan-operation-key>

Representative request:

```
{
  "event_id": "evt_...",
  "credential": "opaque-qr-payload",
  "gate_reference": "gate-2"
}
```

Success:

```
{
  "result": "ADMITTED",
  "ticket": {
    "id": "tkr_...",
    "display": {
      "section": "VIP",
      "row": "A",

```

```

        "seat": "12"
    },
    "admitted_at": "2026-09-12T17:20:15Z"
}

```

Distinct duplicate attempt:

```

{
  "result": "TICKET_ALREADY_ADMITTED",
  "previous_admission": {
    "admitted_at": "2026-09-12T17:19:51Z",
    "gate_reference": "gate-1"
  }
}

```

A retry using the **same** idempotency identity as the successful first ScanAttempt returns the original ADMITTED result rather than a misleading duplicate result.

49. Admission Results

Representative machine outcomes include:

- ADMITTED;
- TICKET_ALREADY_ADMITTED;
- TICKET_INVALID;
- TICKET_VOID;
- CREDENTIAL_REVOKED;
- CREDENTIAL_SUPERSEDED;
- WRONG_EVENT;
- EVENT_CANCELLED;
- ADMISSION_NOT_OPEN;
- NOT_AUTHORIZED;
- AUTHORITY_TEMPORARILY_UNAVAILABLE.

The scanner UI **MUST** distinguish an authoritative rejection from temporary inability to reach authority.

PART V - ADMIN API

50. Admin API Design Rule

Material state transitions are exposed as explicit domain commands, not as generic status patches.

For example:

POST /events/{id}/pause-sales

is preferred over:

PATCH /events/{id} { "state": "PAUSED" }

because lifecycle transitions have distinct authorization, guard, transaction, and audit semantics.

51. Venue and Layout Endpoints

Representative endpoints:

POST /api/v1/admin/venues

GET /api/v1/admin/venues

GET /api/v1/admin/venues/{venue_id}

```
POST /api/v1/admin/venues/{venue_id}/layout-versions
GET /api/v1/admin/venues/{venue_id}/layout-versions
GET /api/v1/admin/venue-layouts/{layout_id}
PATCH /api/v1/admin/venue-layouts/{layout_id}
POST /api/v1/admin/venue-layouts/{layout_id}/publish
```

Only DRAFT layout versions are materially editable.

Published physical layout identity is immutable.

The layout payload may contain normalized components and an opaque third-party geometry object, but the third-party library must not become TktSync's authoritative inventory identity.

52. Event Configuration Endpoints

Representative endpoints:

```
POST /api/v1/admin/events
GET /api/v1/admin/events/{event_id}
PATCH /api/v1/admin/events/{event_id}
POST /api/v1/admin/events/{event_id}/materialize-layout
GET /api/v1/admin/events/{event_id}/inventory
```

Generic PATCH `/events/{id}` is limited to configuration fields whose current Event state permits modification. It MUST NOT be used to perform lifecycle transitions.

Materializing an Event layout creates/updates the Event-specific snapshot and inventory only while governing history rules permit it.

53. Event Transaction Policy Configuration

Representative endpoint:

```
PUT /api/v1/admin/events/{event_id}/transaction-policy
```

It may configure bounded values such as:

- hold duration;
- checkout protection duration;
- payment retry duration;
- reconciliation duration;
- maximum Reservation lifetime;
- maximum hold quantity;
- maximum active Reservations;
- voided-inventory re-release policy.

Configured values MUST remain within platform safety limits.

54. Pricing Endpoints

Representative endpoints:

```
POST /api/v1/admin/events/{event_id}/price-tiers
PATCH /api/v1/admin/events/{event_id}/price-tiers/{price_tier_id}
POST /api/v1/admin/events/{event_id}/pricing/assignments
```

Pricing mutation is ordered against new holds by the authoritative Event transaction rules.

Existing Reservation item snapshots MUST NOT be silently repriced.

55. Block Endpoints

POST /api/v1/admin/events/{event_id}/blocks
POST /api/v1/admin/blocks/{block_id}/release

Create-block requests are all-or-nothing by default.

The API MUST NOT allow an admin to silently block inventory already protected by a buyer Reservation, confirmed Sale, or other incompatible obligation.

Conflicting inventory should be returned in a machine-readable conflict detail where safe.

56. Allocation Endpoints

POST /api/v1/admin/events/{event_id}/allocations
POST /api/v1/admin/allocations/{allocation_id}/release
POST /api/v1/admin/allocations/{allocation_id}/reclassify

Allocation mode is explicit:

- CHANNEL;
- NON_PUBLIC.

Channel allocations identify an authorized Partner.

Non-public allocations do not create commercial Sales merely because tickets are later issued.

Releasing an Allocation MUST NOT displace an active Reservation sourced from that Allocation.

57. Non-Public Issuance

POST /api/v1/admin/allocations/{allocation_id}/issuances

Allowed only for an eligible NON_PUBLIC Allocation.

Representative result:

```
{
  "issuance_id": "iss_...",
  "tickets": [
    {
      "id": "tkr_...",
      "status": "ACTIVE",
      "credential_id": "cred_..."
    }
  ]
}
```

No Sale is created.

Reporting must preserve ISSUED separately from commercial SOLD.

58. Event Lifecycle Commands

Representative endpoints:

POST /api/v1/admin/events/{event_id}/open-sales
POST /api/v1/admin/events/{event_id}/pause-sales
POST /api/v1/admin/events/{event_id}/resume-sales
POST /api/v1/admin/events/{event_id}/close-sales
POST /api/v1/admin/events/{event_id}/cancel
POST /api/v1/admin/events/{event_id}/complete

Each command:

- requires idempotency;
- validates role/Event scope;
- validates legal transition;
- uses server-authoritative state;
- writes audit history.

Cancellation requests **MUST** include an explicit reason.

Cancellation does not synchronously delete or rewrite every Sale/Ticket/Reservation.

59. Ticket Administration

Representative endpoints:

```
POST /api/v1/admin/tickets/{ticket_id}/void
POST /api/v1/admin/tickets/{ticket_id}/credentials/reissue
GET  /api/v1/admin/tickets/{ticket_id}/credential
POST /api/v1/admin/tickets/{ticket_id}/inventory/re-release
```

58.1 Void

Voids current entitlement and revokes the active credential.

It does not erase Sale/Issuance history and does not automatically free inventory.

58.2 Credential Reissue

Creates a replacement active credential while preserving Ticket identity.

58.3 Inventory Re-release

Is a distinct privileged operation.

It requires:

- voided entitlement where applicable;
 - no active consuming entitlement/Reservation conflict;
 - Event policy allowing re-release;
 - explicit safe destination;
 - reason and audit.
-

60. Admission Correction and Manual Override

Representative endpoints:

```
POST /api/v1/admin/admissions/{admission_id}/reverse
POST /api/v1/admin/admissions/manual-override
```

These commands require Gate Supervisor or higher authority as defined by policy.

They **MUST**:

- require a reason;
 - preserve original ScanAttempt history;
 - not delete the original Admission;
 - not create two simultaneous active Admissions for one single-entry Ticket.
-

61. Partner Administration

Self-service Partner onboarding is outside MVP, but Platform Admin requires operational endpoints to configure the 2-3 assessment integrations.

Representative endpoints:

```
POST /api/v1/admin/partners
POST /api/v1/admin/partners/{partner_id}/credentials
POST /api/v1/admin/partner-credentials/{credential_id}/revoke
POST /api/v1/admin/partners/{partner_id}/disable
POST /api/v1/admin/partners/{partner_id}/enable
POST /api/v1/admin/events/{event_id}/partners/{partner_id}/access
POST /api/v1/admin/events/{event_id}/partners/{partner_id}/access/disable
```

Operational Partner disable and security credential revocation are separate commands.

Neither command deletes historical Reservations/Sales.

62. Admin Reporting and Audit

Representative endpoints:

```
GET /api/v1/admin/events/{event_id}/reports/inventory
GET /api/v1/admin/events/{event_id}/reports/sales
GET /api/v1/admin/events/{event_id}/reports/admission
GET /api/v1/admin/events/{event_id}/accreditation-export
GET /api/v1/admin/events/{event_id}/audit
```

Reports and exports are derived read surfaces.

They MUST NOT mutate authoritative inventory, Ticket, Reservation, or Admission state.

PART VI - PARTNER INTEGRATION OPERATING RULES

63. Normal Commercial Flow

1. Partner reads Event/layout/availability.
2. Buyer selects exact returned offers.
3. Partner creates one atomic Reservation.
4. TktSync returns Reservation + authoritative deadline + continuation token.
5. Buyer decides whether to purchase.
6. Before chargeable payment, Partner calls begin checkout.
7. TktSync returns COMMITTING + CheckoutAttempt.
8. Partner performs payment.
- 9a. Success -> Partner confirms using same transaction identity.
- 9b. Definitive failure -> Partner reports payment failure.
- 9c. Unknown outcome -> Partner does not falsely release; TktSync reconciliation rules apply
10. Confirmation returns one Sale and Ticket identities.
11. Partner retrieves active QR credentials for customer communication.

This flow is mandatory for compatible integrations.

64. Stale Availability

When availability becomes stale and hold creation loses a race:

- TktSync returns an inventory conflict;
- no partial hold survives;
- Partner refreshes availability;
- Partner obtains customer approval for any replacement selection;

- new intent uses a new idempotency identity.

TktSync MUST NOT silently substitute another seat, GA source, price, or Allocation.

65. Payment Timeout / Network Ambiguity

64.1 Partner -> TktSync Timeout

If Partner does not receive a response from a TktSync mutation:

- do not assume failure;
- retry with the same idempotency key;
- or query authoritative Reservation status.

64.2 Partner -> Payment Provider Timeout

If payment outcome is unknown:

- do not call payment-failure merely to free inventory;
 - do not create a second independent TktSync Reservation for the same customer payment;
 - allow the protected transaction to reconcile;
 - confirm if valid payment success becomes known before reconciliation expiry;
 - definitively fail/release only when payment safety is established.
-

66. Confirmation After Reconciliation Expiry

Once authoritative reconciliation expiry has released inventory, ordinary confirm MUST fail.

A late payment does not permit the Partner to silently reclaim the original seat or GA quantity.

Any future exceptional recovery must be a distinct privileged workflow that atomically reacquires inventory if permitted and remains auditable.

Financial/customer remediation remains the Partner's responsibility if inventory can no longer be fulfilled.

67. Partner Operational Disable

After operational disable commits:

- new inventory acquisition fails;
- inventory-expanding modifications fail;
- already accepted legitimate Reservations may continue under their existing windows when policy permits;
- credential authentication may remain valid unless credentials are separately revoked.

The API MUST NOT silently equate PARTNER_DISABLED with invalid credentials.

68. Credential Security Revocation

A revoked Partner credential cannot authenticate.

Credential revocation by itself does not erase pre-existing Reservation rights.

An alternative valid credential or privileged recovery process may continue eligible transactions where policy permits.

69. Event Pause and Sales Close

For Event PAUSED or SALES_CLOSED:

- new Reservations are rejected;
- valid existing Reservations retain their existing deadline rights;
- existing valid Reservations may begin protected checkout according to governing policy;
- in-flight committing/reconciling transactions may resolve within already-authorized windows.

The API MUST distinguish these from Event cancellation.

70. Event Cancellation

After Event cancellation commits:

- new Reservations fail;
- new checkout protection fails;
- new commercial confirmation accepted after cancellation fails;
- ordinary admission fails;
- confirmed Sale/Ticket history remains visible;
- Partner refunds/customer remediation remain outside TktSync payment authority.

If a payment was already in progress when cancellation won the authoritative ordering, that transaction follows the cancellation-aware reconciliation rules rather than silently becoming a new Sale.

71. Partner Reference Semantics

Fields such as:

- partner_customer_ref;
- partner_order_ref;
- partner_payment_ref;
- buyer_session_ref

are opaque Partner correlation values.

They are not universal idempotency keys.

The API does not impose a universal global uniqueness rule on these fields unless a Partner-specific integration profile explicitly guarantees one.

Partners SHOULD use stable values that allow their own support/reconciliation workflows to map TktSync records back to Partner records.

PART VII - ERROR MODEL

72. Core Business Error Codes

Inventory / Reservation

- INVENTORY_UNAVAILABLE
- INSUFFICIENT_GA_QUANTITY
- INVENTORY_NOT_ELIGIBLE_FOR_PARTNER
- CURRENCY_MISMATCH
- HOLD_EXPIRED
- HOLD_NOT_OWNED
- RESERVATION_NOT_MODIFIABLE
- CHECKOUT_ALREADY_ACTIVE
- CHECKOUT_WINDOW_EXPIRED
- PAYMENT_STATUS_UNCERTAIN
- RECONCILIATION_EXPIRED

- `ALREADY_CONFIRMED`
- `IDEMPOTENCY_CONFLICT`

Event / Partner

- `EVENT_NOT_ON_SALE`
- `EVENT_PAUSED`
- `EVENT_SALES_CLOSED`
- `EVENT_CANCELLED`
- `EVENT_COMPLETED`
- `PARTNER_DISABLED`
- `PARTNER_EVENT_ACCESS_DISABLED`
- `NOT_AUTHORIZED`

Ticket / Admission

- `TICKET_INVALID`
- `TICKET_VOID`
- `CREDENTIAL_REVOKED`
- `CREDENTIAL_SUPERSEDED`
- `TICKET_ALREADY_ADMITTED`
- `ADMISSION_NOT_OPEN`
- `WRONG_EVENT`
- `AUTHORITY_TEMPORARILY_UNAVAILABLE`

Protocol / Validation

- `VALIDATION_ERROR`
- `RESOURCE_NOT_FOUND`
- `RATE_LIMITED`
- `INTERNAL_ERROR`

The upstream domain list is a minimum stable vocabulary; this API adds only transport/interface-specific distinctions needed to make client recovery deterministic.

73. Retry Guidance by Error

- **`INVENTORY_UNAVAILABLE`** - refresh availability, obtain a new customer selection, and use a new idempotency key for the changed intent.
 - **`INSUFFICIENT_GA_QUANTITY`** - refresh quantity, obtain the customer's new quantity choice, and use a new idempotency key.
 - **`HOLD_EXPIRED`** - do not charge; begin a new customer-approved Reservation.
 - **`CHECKOUT_WINDOW_EXPIRED`** - read the Reservation and follow its authoritative state; do not assume payment safety.
 - **`PAYMENT_STATUS_UNCERTAIN`** - reconcile payment; do not free inventory blindly.
 - **`RECONCILIATION_EXPIRED`** - do not ordinary-confirm; use explicit recovery/remediation if permitted.
 - **`ALREADY_CONFIRMED`** - read the existing Sale/Tickets; do not create another Sale.
 - **`IDEMPOTENCY_CONFLICT`** - correct caller key reuse; do not retry altered intent under the same key.
 - **`PARTNER_DISABLED`** - stop new acquisition and support only existing eligible transactions.
 - **`AUTHORITY_TEMPORARILY_UNAVAILABLE`** - retry safely; preserve the same idempotency key for the same mutation intent.
 - **`RATE_LIMITED`** - respect Retry-After; keep the same key if retrying the same mutation intent.
-

PART VIII - RETRIES, RATE LIMITING, AND FAILURE BEHAVIOR

74. Retry Safety Matrix

- **GET reads** - may be retried.
- **State-changing request with no received response** - retry using the same idempotency key.
- **State-changing request whose intent has changed** - use a new idempotency key.
- **Confirm after timeout** - retry with the same key or read authoritative Reservation status.
- **Scan after timeout** - retry with the same scan idempotency key.
- **Availability conflict** - refresh and submit the customer's new intent; do not reuse the old key for altered content.

Automatic retry middleware **MUST NOT** generate a new idempotency key for the same logical mutation.

75. Rate Limiting

Rate limits are configurable and may vary by Partner, operation, and environment.

When rate limited:

- HTTP status is 429;
- business code is RATE_LIMITED;
- Retry-After SHOULD be returned where useful.

Rate limiting **MUST NOT** create or alter inventory ownership.

Anti-hoarding controls may additionally restrict:

- quantity per hold;
 - active Reservations per Partner;
 - active Reservations per buyer/session;
 - repeated mutation patterns.
-

76. Authority Failure

If the authoritative PostgreSQL state cannot be safely evaluated:

- hold creation fails closed;
- confirmation does not guess;
- admission does not claim duplicate-safe authority;
- stale availability is not converted into ownership.

Representative outcome:

AUTHORITY_TEMPORARILY_UNAVAILABLE

Temporary unavailability is preferable to overselling or contradictory entitlement.

PART IX - SECURITY BOUNDARIES AT THE API CONTRACT

77. Least Authority

The API **MUST** enforce least authority for every surface.

Partner credentials cannot become admin credentials.

BuyerSelectionSession capabilities cannot become Partner credentials.

Scanner credentials cannot become Event-management credentials.

78. Token Transport

Secrets/capabilities MUST NOT be placed in:

- query parameters;
- analytics event names;
- general application logs;
- client-visible error messages.

Opaque selection URLs are an exception only insofar as their internal bootstrap mechanism is specifically designed by the Security/Auth specification to prevent normal referrer/history leakage of bearer material.

Reservation continuation tokens SHOULD be transported in headers or secure browser POST handoff fields, not query strings.

79. Cross-Partner Privacy

Partner API responses MUST NOT expose:

- another Partner's Reservation IDs;
- another Partner's order/payment references;
- another Partner's customer/session references;
- reason-specific inventory state that leaks another Partner's transaction;
- another Partner's channel Allocation configuration unless explicitly shared by Event policy.

Caller-contextual availability may differ across Partners while still representing one authoritative inventory truth.

80. PII Minimization

TktSync inventory ownership does not require unrelated customer PII.

Partner references are preferred.

Where attendee/accreditation data is required, it is exposed only to authorized surfaces and kept separate from core inventory responses.

QR payloads SHOULD contain no unnecessary customer PII.

PART X - CONTRACT GENERATION AND CLIENTS

81. OpenAPI

The production HTTP API SHALL have a versioned OpenAPI 3.1 description representing this contract.

The OpenAPI definition MUST be generated from or validated against the same Go API contract used by the implementation.

It MUST NOT be maintained as an unreviewed parallel truth.

82. TypeScript Client Generation

The React/Vite applications and supported Partner SDK tooling SHOULD consume generated TypeScript contracts/clients from the versioned OpenAPI specification.

This replaces direct language-level type sharing between Go and TypeScript.

The generated client is a transport convenience; it does not replace this normative behavioral contract.

PART XI - ASYNCHRONOUS NOTIFICATION BOUNDARY

83. What This Document Does Not Define

This contract intentionally does not define the asynchronous Partner notification mechanism.

The subsequent Realtime / Event Contract will define:

- publishable domain facts;
- realtime channel authorization;
- event sequence/revision behavior;
- reconnect/resync semantics;
- webhook behavior if webhooks are included;
- delivery retry/acknowledgement semantics;
- sanitized payload schemas.

No asynchronous notification may redefine the command/query semantics in this API contract.

Realtime or webhook delivery remains freshness/integration support, not inventory authority.

PART XII - CROSS-DOCUMENT TRACEABILITY

84. Technical Brief Alignment

- **Lightweight Partner API** - Partner /api/v1 surface.
- **Availability** - Event availability endpoint.
- **Cached high-traffic availability** - freshness metadata while reads remain non-authoritative.
- **Atomic Hold** - POST /partner/reservations is all-or-nothing.
- **Hold token and expiry** - Reservation continuation token plus authoritative deadline fields.
- **Concurrent same-unit rejection** - machine-readable inventory conflict outcomes.
- **Confirm / Release** - explicit Reservation command endpoints.
- **QR issuance** - Ticket credential creation plus authorized Partner/Admin retrieval.
- **White-label seat selection** - BuyerSelectionSession, Selection API, and secure Partner hand-off.
- **Partner retains checkout/payment** - checkout protection first, Partner payment second, TktSync confirmation third.
- **Multiple Partners** - Partner authentication, Event access, and caller-contextual availability.
- **GA / Reserved / Mixed** - offer model plus atomic mixed Reservation items.
- **Allocations / blocks** - Admin block/allocation commands.
- **Duplicate scan prevention** - Admission scan endpoint with idempotent-vs-duplicate semantics.
- **Partner reporting** - Partner-scoped reporting endpoints.
- **Accreditation export** - Admin derived-export endpoint.

The contract preserves the brief's responsibility boundary: TktSync owns inventory, confirmation, ticket/QR validity, duplicate scan prevention, audit, and reporting; Partners own payment/customer relationship; Event owners own physical operations.

85. Platform Policy Alignment

- **Availability is not ownership** - explicit warning; only successful Reservation creation grants temporary ownership.
- **Atomic acquisition** - hold requests are all-or-nothing.
- **No silent substitution** - offer-specific selection; no hidden source/seat replacement.
- **Server time authoritative** - deadline fields plus server_time; client timers remain advisory.
- **Hold ownership scoped** - Partner/session authorization plus Reservation continuation token.
- **Protected checkout before payment** - explicit /checkout Partner obligation.

- **Bounded retry/reconciliation** - canonical states and deadlines are exposed.
 - **Unknown payment protected** - payment-failure reporting requires a definitive outcome.
 - **Confirmation exactly once** - required idempotency plus existing-Sale recovery.
 - **Release source-aware** - Partner never chooses the internal return destination.
 - **Price snapshot** - Reservation items expose frozen Event-controlled terms.
 - **Admin cannot steal a hold** - Admin commands return conflicts instead of silently overriding accepted buyer rights.
 - **Partner neutrality** - caller-contextual offers without a hidden priority API.
 - **Operational disable vs credential revoke** - separate errors and commands.
 - **Ticket / QR separation** - separate Ticket and credential resources.
 - **Partner-owned Ticket void/reissue** - Partner-scoped commands are limited to the Partner's own commercial Tickets.
 - **Scan retry vs duplicate** - same scan key replays; a distinct scan reports duplicate/already-admitted.
 - **Void separate from resale** - separate Ticket void and inventory re-release commands.
 - **Reports derived** - report/export endpoints remain read-only projections.
 - **PII minimized** - Partner references and limited ticket display fields are preferred.
-

86. Logical Domain Alignment

The API preserves separate canonical dimensions:

- Event state;
- Reservation state;
- Ticket state;
- credential state;
- Admission result/state;
- Partner account/access state.

The API does not collapse these into a generic status shared across unrelated resources.

SOLD is represented by confirmed Sale/inventory semantics and MUST NOT be inferred from COMMITTING or external payment success alone.

ISSUED remains distinct from commercial Sale through the Admin non-public issuance API.

87. System Architecture Alignment

- **One Go authoritative backend** - all four API surfaces route to the same authoritative domain implementation.
 - **PostgreSQL transactional authority** - the API never promises ownership from cache or real-time state.
 - **Durable transactional idempotency** - Idempotency-Key is required for retrievable mutations.
 - **Worker not authoritative for expiry** - API effective-state guards do not trust a stale persisted status label alone.
 - **Event lifecycle gate** - lifecycle commands and concurrent Partner commands have deterministic ordering.
 - **Confirmation in one transaction** - the API returns one committed Sale/Ticket result.
 - **Admission in one transaction** - scan response corresponds to the authoritative ScanAttempt/Admission outcome.
 - **No external calls in core transaction** - Partner payment occurs outside the TktSync database transaction after checkout protection.
 - **Transactional outbox** - asynchronous notifications are post-commit and deferred to the Event contract.
-

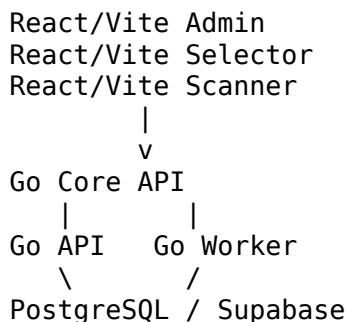
88. Relational Schema Alignment

- **Reservation continuation token stored as digest** - API returns an opaque token and requires replay-safe recovery.

- **Partner references have no universal uniqueness rule** - API treats them as opaque correlation values.
 - **One Reservation currency** - hold rejects incoherent mixed-currency commercial selection.
 - **ReservationItem source/price snapshot** - API exposes immutable item price without internal source IDs.
 - **GA shared/allocation buckets** - API exposes opaque caller-eligible offers rather than bucket internals.
 - **One Sale per Reservation** - confirm returns exactly one Sale.
 - **TicketEntitlement separate from QRCredential** - Ticket and credential have separate representations/endpoints.
 - **One active QR per Ticket** - credential retrieval resolves the current active credential.
 - **One active Admission** - scan processing produces exactly one distinct authoritative admission winner.
 - **Append-only audit/outbox facts** - Admin/Partner reads never mutate historical facts.
-

89. Technology Stack Alignment

The contract supports the approved monorepo implementation:



OpenAPI is the language-independent boundary between Go and React/TypeScript.

No API surface requires a second authoritative backend runtime.

PART XIII - REVIEW FINDINGS AND DRIFT RESOLUTIONS

90. Review Finding: API Must Preserve Source-Specific Acquisition Without Exposing Allocation IDs

A naive GA hold API using only `inventory_id` can lose source intent when a caller is simultaneously eligible for shared quantity and its own channel Allocation.

The governing pricing policy does not give Allocations a separate price override; the issue is acquisition source and restoration semantics, not different Allocation pricing.

Resolution: availability may expose opaque `offer_id` values representing caller-eligible acquisition sources. Hold requests select the returned offer when source distinction is required. Internal Allocation IDs remain private while source-aware release remains deterministic.

This prevents silent source substitution without inventing Allocation-specific pricing semantics.

91. Review Finding: Token Recovery Must Survive Lost Responses

The schema stores token digests rather than ordinary plaintext continuation tokens. Idempotency requires a lost successful response to be recoverable.

Resolution: this contract requires replay-safe recovery of the same usable continuation token/equivalent through the future Security/Auth design. The implementation cannot simply generate an unrecoverable random token and discard it before the caller has safely received it.

The same concern applies to retrieving active QR credential material after a lost confirmation response.

92. Review Finding: Confirm Should Not Inline Irrecoverable QR Secrets

Returning a one-time raw QR value only inside the confirmation response would make response loss operationally unsafe.

Resolution: confirmation returns stable Ticket/Credential identities. A separate authorized credential endpoint retrieves the current active QR representation without creating a new Ticket or silently rotating the credential.

93. Review Finding: Definitive Payment Failure Must Be Distinct From Timeout

A generic release call during payment could allow a Partner to free inventory while payment is uncertain.

Resolution: payment-failure explicitly asserts a definitive failure/non-charge outcome for a CheckoutAttempt. Ordinary release from uncertain COMMITTING/RECONCILING state is rejected with PAYMENT_STATUS_UNCERTAIN.

94. Review Finding: Partner Order References Are Not Idempotency Keys

The relational model intentionally does not assign universal uniqueness semantics to Partner order/payment references.

Resolution: the API keeps idempotency identity separate from Partner correlation fields. Partner-specific integration profiles may impose stronger local rules without making them global platform semantics.

95. Review Finding: Admin Lifecycle Must Not Be Generic PATCH

Generic state patches would bypass command-specific guards/audit requirements.

Resolution: Event lifecycle, ticket void, credential rotation, inventory re-release, Allocation release, and Admission correction use explicit command endpoints.

96. Review Finding: White-Label Return Must Not Leak Hold Tokens

The brief requires return to Partner checkout with the acquired hold. Putting bearer/continuation material in query parameters risks browser history/referrer leakage.

Resolution: the selection handoff SHOULD use an HTTPS form POST or another Security-approved opaque handoff; the Partner return destination is pre-registered.

97. Review Finding: Async Notifications Must Not Be Mixed Into Command Semantics

Webhook/realtime behavior has different authorization, delivery, replay, and ordering concerns from synchronous HTTP commands.

Resolution: asynchronous notification contracts remain a separate governing specification. This API defines authoritative command/query results only.

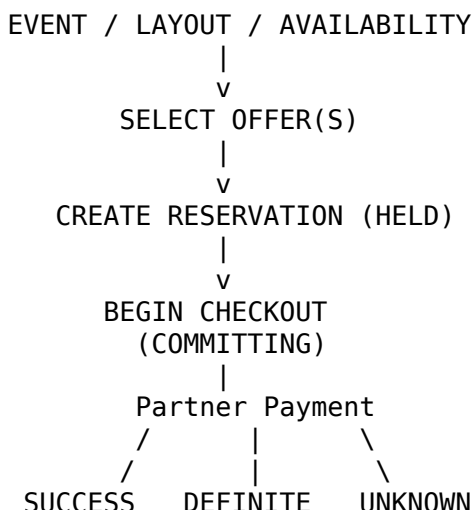
PART XIV - NON-NEGOTIABLE API INVARIANTS

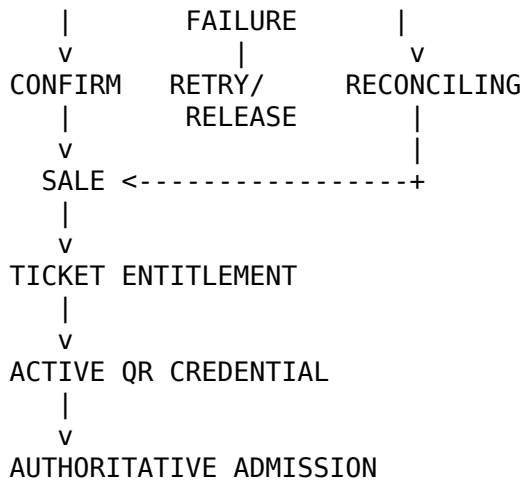
98. API Invariants

1. Availability never creates inventory ownership.
2. A successful Reservation is the normal pre-sale ownership boundary.
3. Multi-item Reservation creation is all-or-nothing.
4. Clients select explicit offers; TktSync does not silently substitute inventory, source, quantity, or price.
5. All Reservation deadlines are server-authoritative.
6. Partner payment begins only after checkout protection succeeds.
7. Unknown payment outcome is not reported as definitive failure.
8. Reconciliation remains visible and bounded.
9. Confirmation is idempotent and creates exactly one Sale.
10. A lost confirmation response is recoverable without duplicate Sale/Ticket creation.
11. Partner release never specifies the internal inventory destination.
12. Commercial SOLD is never inferred from external payment alone.
13. Non-public ISSUED inventory never masquerades as a Sale.
14. Ticket identity is separate from credential identity.
15. Active credential retrieval does not silently create a replacement credential.
16. Ticket void does not imply inventory resale.
17. Partner ticket void/reissue is limited to Tickets originating from that Partner's commercial Sales; re-release remains Event-policy dependent.
18. Scan retry and distinct duplicate scan remain different outcomes.
19. BuyerSelectionSession capability cannot confirm Sales or access Partner secrets.
20. Partner credentials cannot administer another Partner's private transaction.
21. Admin authority cannot silently displace accepted buyer rights.
22. Event cancellation prevents later new Sale confirmation but preserves history.
23. Business errors remain machine-readable and stable enough for deterministic recovery.
24. HTTP status is not the sole business signal.
25. All externally retrievable mutations use durable idempotency.
26. Same idempotency key with changed intent is rejected.
27. Partner/customer correlation fields are not substitutes for idempotency.
28. Tokens and credentials are not transported through unsafe query parameters.
29. Realtime/webhook messages cannot create ownership or override API/database authority.
30. When authoritative state cannot be established, irreversible operations fail closed.
31. The production OpenAPI contract and generated clients must preserve these invariants.

99. Final Contract Summary

The canonical Partner integration is:





The governing integration principle is:

Partners own checkout and payment. TktSync owns the inventory right being purchased, the authoritative Sale confirmation, Ticket/credential validity, and admission truth. Every API operation preserves that boundary.

End of Document